Module 08 — Data Structures Overview

Phase: Core Python
Previous: Dictionaries
Next: Functions, Decorators, and Generators

Module Outcome

Choose list vs tuple vs set vs dict for a job, spot a wrong container, and combine them in one small report.

This module is a **decision capstone**, not a re-teach. If syntax feels fuzzy, revise that chapter first — then return here to **choose**.

By the end of this module you will be able to:

- Pick a container by the job (lookup, unique, sealed, ordered-editable)
- Say when **not** to use each one
- Combine a list of dicts with a set and a totals dict
- Justify a choice in one sentence (structure-chooser mindset)

---

1. Why This Concept Matters

You already know four bags. The new skill is **which bag**.

A cart that must stay in order and grow is a list. A GPS drop that must not change is a tuple. Unique coupon codes are a set. Price by SKU is a dict.

Wrong container → slow checks, accidental edits, or awkward code. Right container → the program reads as the job.

```
Need key → value lookup?       → dict
Need unique items only?        → set
Need fixed record (no edits)?  → tuple
Need ordered, changeable list? → list
```

---

2. The decision guide

Understand

Four built-in containers, four jobs. Pick by the job, not by habit.

Ask four questions:

1. Do I look up by a **key**? → **dict**
2. Do I only care about **uniqueness** / membership? → **set**
3. Should this row stay **frozen**? → **tuple**
4. Do I need an **ordered, editable** sequence? → **list**

```
Shopping bag (list)     — add/remove items, order matters
Sealed envelope (tuple) — receipt line that must not change
```

```
Guest list (set)        — each person once; “are they in?”
Filing cabinet (dict)   — label → folder
```

Structure	Example	Best when
List	`[101, 102, 105]`	Order matters; duplicates OK
Tuple	`(101, "Ananya", "Eng")`	Fixed row; sealed values
Set	`{101, 102, 105}`	Unique + fast `in`
Dict	`{101: "Ananya"}`	Lookup by key

How It Works

Same campus canteen, four jobs — not four syntax lessons:

```
cart = ["Idli", "Filter coffee"]
drop_point = (12.97, 77.59)
categories = {"South", "Beverage"}
sku_price = {"IDLI01": 40, "COFF02": 25}

print(cart[0])
print(drop_point[0])
print("South" in categories)
print(sku_price["IDLI01"])
```

Expected Output:

```
Idli
12.97
True
40
```

⚠️ WATCH OUT

```
# Wrong mental model
# "I'll always use a list."

# Prefer
# lookup -> dict; unique -> set; sealed -> tuple; sequence -> list
```

Try It

For each need, name the structure only: (a) undo history of form fields, (b) employee name by ID, (c) unique coupon codes, (d) fixed GPS pair.

---

3. When not to use each

Understand

Anti-patterns matter as much as the happy path. Spot a wrong container before it causes bugs in reviews and interviews.

- Huge list + repeated `in` → slow scan — prefer a **set**
- Dict with fake index keys `"0"`, `"1"` → prefer a **list**
- Mutable list for fixed config someone might `.append` → prefer a **tuple**

How It Works

```
banned = {101, 102, 105}
if 101 in banned:
    print("Blocked")

names = ["A", "B"]
print(names[0])

API_VERSION = ("v2",)
print(API_VERSION)
```

Expected Output:

```
Blocked
A
('v2',)
```

⚠️ WATCH OUT

```
# Slow membership on a large collection
if user_id in big_list:       # scans each time

# Prefer
if user_id in id_set:

# Fake positions
{"0": "A", "1": "B"}          # that is not a list
```

Try It

Rewrite in one sentence: “Store 10,000 product SKUs only to answer: is this SKU on sale?” Which container, and why?

---

4. Combine structures

Understand

Production data is rarely one container. Orders are often a **list of dicts**; unique SKUs use a **set**; category totals use a **dict**.

```
orders: list
   └── each order: dict
          ├── id, customer
          └── items: list of dicts (sku, qty, price)
```

If `.get` or nested `for` feels new, revisit Dictionaries and Lists — then come back. Prefer a normal `for` (no comprehension required).

How It Works

```
orders = [
    {
        "id": "ORD-1",
        "customer": "Priya",
        "items": [
            {"sku": "PEN", "qty": 2, "price": 10},
            {"sku": "NBK", "qty": 1, "price": 40},
        ],
    },
    {
        "id": "ORD-2",
        "customer": "Ravi",
        "items": [
            {"sku": "PEN", "qty": 1, "price": 10},
        ],
    },
]

skus = set()
revenue = 0
for order in orders:
    for item in order["items"]:
        skus.add(item["sku"])
        revenue += item["qty"] * item["price"]

print("Unique SKUs:", sorted(skus))
print(f"Revenue: ₹{revenue}")
```

Expected Output:

```
Unique SKUs: ['NBK', 'PEN']
Revenue: ₹70
```

Category totals are the same idea on a flat list of dicts: loop, `set.add` for unique cats, `totals.get(cat, 0) + amount` for the running dict. Practice below uses that shape.

⚠ WATCH OUT

Do not re-learn list/dict syntax here. The skill is **which bag holds which fact** — lines in a list, lookup in a dict, uniqueness in a set.

Try It

Given `cart = [{"sku": "TEA", "qty": 2, "price": 15}, {"sku": "SAM", "qty": 1, "price": 25}]`, print total payable and the sorted set of SKUs.

Step-by-Step: Build It

One assemble: choose containers for a canteen cart, then compute payable + unique SKUs. Say **why** in a comment when you open the kit.

```
cart_lines = [                          # list — ordered, editable lines
    {"sku": "TEA", "qty": 2, "price": 15},
    {"sku": "SAM", "qty": 1, "price": 25},
```

```
]
drop_point = (12.97, 77.59)             # tuple — sealed GPS
on_sale = {"TEA", "SAM", "NBK"}         # set — membership
menu_price = {"TEA": 15, "SAM": 25}     # dict — lookup by SKU

total = 0
skus = set()
for line in cart_lines:
    total += line["qty"] * line["price"]
    skus.add(line["sku"])

print(f"Drop: {drop_point}")
print("TEA on sale?", "TEA" in on_sale)
print("Menu TEA ₹", menu_price["TEA"])
print("Payable ₹", total)
print("SKUs:", sorted(skus))
```

Expected Output:

```
Drop: (12.97, 77.59)
TEA on sale? True
Menu TEA ₹ 15
Payable ₹ 55
SKUs: ['SAM', 'TEA']
```

A list `append` / `pop` is also the everyday **undo** model (last in, first out). The kit asks you to practise that stack — not to re-teach lists.

Practice Exercises

Attempt each problem before reading the model answer.

1. For each need, name the structure only: (a) undo history of form fields, (b) employee name by ID, (c) unique coupon codes, (d) fixed API version string pair.

Model answer:

```
(a) list
(b) dict
(c) set
(d) tuple
```

2. Rewrite correctly: “Store 10,000 product SKUs only to answer: is this SKU on sale?”

Hint: Membership at that size should not scan a list.

Model answer:

Use a **set** (or dict keys) for membership — not a plain list scan.

```
on_sale = {"SKU-1", "SKU-9", "SKU-42"}
print("SKU-9" in on_sale)
```

Expected Output:

```
True
```

3. Given `cart = [{"sku": "TEA", "qty": 2, "price": 15}, {"sku": "SAM", "qty": 1, "price": 25}]`, print total payable and the sorted set of SKUs.

Model answer:

```
cart = [
    {"sku": "TEA", "qty": 2, "price": 15},
    {"sku": "SAM", "qty": 1, "price": 25},
]
total = 0
skus = set()
for line in cart:
    total += line["qty"] * line["price"]
    skus.add(line["sku"])
print(total)
print(sorted(skus))
```

Expected Output:

```
55
['SAM', 'TEA']
```

4. Why is `names = {"0": "A", "1": "B"}` a poor stand-in for an ordered name list? What should you use instead?

Model answer:

A dict is for **key → value** lookup, not for fake positions. Use a **list**: `names = ["A", "B"]` so order and indexing stay natural.

5. Hard-code three expense dicts. Build category totals with `.get` (no comprehension). Print Food and Travel totals.

Model answer:

```
expenses = [
    {"id": 1, "cat": "Food", "amount": 120},
    {"id": 2, "cat": "Travel", "amount": 80},
    {"id": 3, "cat": "Food", "amount": 60},
]
totals = {}
for e in expenses:
    totals[e["cat"]] = totals.get(e["cat"], 0) + e["amount"]
print("Food:", totals["Food"])
print("Travel:", totals["Travel"])
```

Expected Output:

```
Food: 180
Travel: 80
```

Lab

Choose containers and combine them. No complete solution here. This is **not** the kit quiz — it is one combined report. The kit is the mini-project.

Problem

A canteen logs a few expense (or order) lines. You must pick the right bags and print a short ₹ report.

What to Build

A short script with hard-coded lines plus a one-line comment on each container choice.

Requirements

- A **list** of 3–5 dicts (`id`, `cat`, `amount` — or cart lines with `sku` / `qty` / `price`)
- A **set** of categories (or SKUs)
- A **dict** of category (or SKU) totals using `.get`
- Print categories and each total, then a grand total
- Include one rejected idea in a comment (e.g. "not a list of 10,000 SKUs for `in`") and say why

Expected Behavior

Totals add correctly. Missing categories are not assumed — you build them from the data.

Sample Run

```
Categories: Food, Travel
Food: ₹180
Travel: ₹80
Grand: ₹260
```

(Your seed data may differ; the **shape** should match.)

Constraints

Top-level statements only — no `def`, no type hints, no files, no frameworks.

Bonus

Reject duplicate expense IDs using a set while loading. Practise undo: `append` two action strings onto a list, `pop` the last, print it (LIFO).

Mini-Project (Assignment): Structure Chooser + Undo Stack

Why this matters: Code review asks "why a list here?" as often as "does it run?" Choosing well is the skill; `append` / `pop` undo is the same list used as a stack.

Lab kit: 15_Lab_Assets/12_structure_chooser/

File	What you do
`starter/choose.py`	Fill each answer with `"list"` | `"tuple"` | `"set"` | `"dict"`; implement a small undo stack with `append` / `pop`
`broken/`	Fix intentional wrong choices if present
`solution/`	Do **not** copy this

This is **not** the Lab report above. Same decision skill, kit-shaped ticket.

Objective

For each kit scenario, pick the container and justify in one line. Then prove undo with a list stack.

Story / Scenario

A reviewer walks four cases: an ordered cart, a fixed RGB / GPS-style triple, unique ids, and roll → name. You name the bag and push/pop an undo list.

Requirements

Piece	Detail
Chooser	Complete kit scenarios (cart, RGB / GPS-style fixed values, unique ids, roll → name)
Why	One-sentence comment next to each choice
Undo	List as a stack: `append` two actions, `pop` the last (LIFO) — kit requirement
Style	No `def` required; no type hints

Expected Behavior

- Quiz answers match the kit intent (`list` / `tuple` / `set` / `dict`)
- Undo prints the last action pushed
- Quit / script end is clean (no extra framework)

Sample Run

```
{'shopping cart ordered items': 'list', 'RGB color fixed 3 numbers': 'tuple', 'unique employee IDs': 'se
typed email
```

(Your printed last-undo text may differ; LIFO must hold.)

Bonus Challenges

- Add one new scenario of your own (e.g. “banned handles”) and choose the structure
- **Later — Functions week:** wrap “build totals” in a `def`

Done when

- [ ] Quiz answers match the kit intent (list / tuple / set / dict)
- [ ] Each choice has a one-line why
- [ ] Undo stack push/pop works (`append` / `pop`)
- [ ] No type hints on the core path

Interview Q&A

Q1. List vs tuple?
List is mutable and for sequences you change. Tuple is immutable — fixed records, safer config.

Q2. When do you use a set?
Uniqueness and fast membership (`in`). Not for ordered rows that must keep duplicates.

Q3. When do you use a dict?
Key → value lookup (ID → name, SKU → price).

Q4. Cart of line items — what shape?
Usually a **list of dicts** (and sometimes nested lists inside).

Q5. Why not a list for 10,000 banned IDs?
`in` on a list scans each time (O(n)); a **set** (or dict keys) is typically O(1) average — the usual choice at scale.

Q6. Can a list be a dict key?
No — keys must be hashable. Use a tuple for a composite key if needed.

Q7. What is a stack (LIFO) in one line?
Last in, first out — undo / browser Back. A Python **list** with `append` / `pop()` is the everyday teaching model.

Q8. What is a queue (FIFO) in one line?
First in, first out — support tickets. `list.pop(0)` works but shifts items (O(n)); production code often uses `collections.deque`. Know the **idea**; master deque later if needed.

Q9. Big-O one-liner for interviews?
“I use a set or dict when I need fast membership or lookup; I use a list when order and duplicates matter.”

Cheat Sheet

```
list   → ordered, editable; duplicates OK
tuple  → ordered, sealed; hashable if items are
set    → unique, unordered; fast in
dict   → key → value lookup

lookup?     dict
unique?     set
frozen row? tuple
sequence?   list

wrong: list for 10k membership scans
wrong: dict with "0","1" as fake indexes
wrong: list for fixed config someone might append to

combine: list of dicts + set of ids + dict of totals
undo:    list append / pop  (LIFO stack)
```

Completion Checklist

```
[ ] I can pick list vs tuple vs set vs dict for a job
[ ] I can name one anti-pattern for each container
[ ] I can combine a list of dicts with a set and a totals dict
[ ] I completed the lab report
[ ] I attempted the structure-chooser kit
[ ] I can justify a choice in one sentence in an interview
```

What's Next?

You can now **choose** a container, not only remember its syntax.

Next, Functions, Decorators, and Generators wrap the logic you have been writing at the top level into reusable `def` blocks — menus and reports without copy-paste.